ST-ROBOT · TÀI LIỆU TỰ HỌC

VEX IQ

Cơ cấu truyền động, thuật toán đường đi và kiểm thử robot VEX IQ.

6 bài học · Lý thuyết, ví dụ, thực hành, kiểm tra và tài liệu tham khảo

Lộ trình học tập

1. Cấu trúc và truyền động
2. Góc quay và đường đi
3. Dừng theo cảm biến
4. Nhiệm vụ sa bàn
5. Quãng đường, vòng quay và hiệu chuẩn
6. Điều khiển phản hồi và giới hạn tốc độ

Cách sử dụng tài liệu

Đọc mục tiêu và kiến thức nền; tự dự đoán trước ví dụ; thực hành theo các bước; ghi lại dữ liệu và giải thích sai lệch. Sau khi tự làm, đối chiếu đáp án và tự kiểm tra. Phòng lab bên ngoài không tự đồng bộ điểm; hoạt động mô phỏng không thay thế kiểm tra an toàn phần cứng thật.

Bài 1/6 · Cấu trúc và truyền động

Dự đoán và điều khiển robot tiến ba ô, rồi nối được chương trình → bộ điều khiển → động cơ → bánh xe.

Chuẩn bị

Chưa cần có robot hoặc biết Python. Biết trái/phải theo hướng robot đang nhìn. Trên bàn lưới, F nghĩa là tiến một ô, R rẽ phải tại chỗ, L rẽ trái tại chỗ; dùng Reset để về trạng thái đầu.

Sau bài học, bạn có thể

1. Dự đoán rồi kiểm tra đích của một chuỗi lệnh tiến ngắn trên lưới.

2. Chỉ đúng vai trò của Brain, động cơ, bánh xe và cảm biến trong sơ đồ.

3. Ghi được vị trí, hướng và phân biệt ô lưới với khoảng cách mm.

Cùng hiểu bài từ đầu

Một robot chuyển động nhờ cả chương trình lẫn cơ cấu. Brain đọc lệnh, động cơ tạo chuyển động và bánh xe truyền chuyển động ra mặt sân. Bài bắt đầu bằng một chuỗi tiến ngắn trên lưới để bạn thấy rõ thứ tự; sau đó mới liên hệ với hai bánh của robot thật.

Hãy nhờ một bạn cầm mũi tên đứng trên các ô gạch. Mỗi thẻ “tiến” là đi một ô theo mũi tên hiện tại; thẻ “rẽ” chỉ xoay người tại ô đang đứng.

Người đưa thẻ giống phần chương trình xác định thứ tự. Người cầm mũi tên giống robot thực hiện lệnh. Muốn đoán ô tiếp theo phải biết cả ô hiện tại và hướng hiện tại.

Giới hạn của ví von: Một ô trên mô hình chỉ là đơn vị của bàn lưới. Nó không tự bằng một số mm trên VEX IQ; robot thật cần cấu hình đúng động cơ, bánh xe và cơ cấu truyền động.

Những từ cần hiểu

1. Brain: Bộ điều khiển lưu và thực thi chương trình của VEX IQ; bản thân nó không tạo lực đẩy bánh xe.

2. Động cơ và bánh xe: Động cơ biến năng lượng điện thành chuyển động quay. Bánh xe tiếp xúc mặt sân để tạo dịch chuyển khi có điều kiện bám phù hợp.

3. Đầu vào và đầu ra: Cảm biến đưa số đo vào chương trình; lệnh tới động cơ là đầu ra. Lệnh yêu cầu không tự chứng minh robot đã đi đúng.

4. Trạng thái · vị trí và hướng: Trên lưới ghi (x, y, hướng). Bài quy ước x sang phải, y xuống dưới; trạng thái đầu là (0,0,đông).

Từ ba lệnh tiến đến mô hình hai bánh

Bắt đầu với: Trên lưới bắt đầu tại (0,0,đông), thực hiện F,F,F. Trong phép tính riêng cho bánh thật, giả sử hai bánh cùng có vận tốc tiếp tuyến 80 mm/s trong 2 s và lăn không trượt.

Theo dõi từng lệnh trên lưới

F thứ nhất đưa tới (1,0,đông), F thứ hai tới (2,0,đông), F thứ ba tới (3,0,đông). Mỗi F dùng hướng ngay lúc đó; không có lệnh rẽ nên hướng giữ nguyên.

Xét vận tốc của cơ cấu hai bánh

Trong mô hình lý tưởng, hai bánh cùng đẩy tiến với 80 mm/s thì tâm robot có v = (80 + 80)/2 = 80 mm/s. Động cơ lắp đối xứng có thể cần cấu hình đảo chiều để hai bánh cùng tạo hướng tiến.

Tính dịch chuyển với đơn vị vật lý

s = v × t = 80 mm/s × 2 s = 160 mm. Đây là kết quả của giả định bánh thật, không phải phép đổi từ 3 ô thành 160 mm.

Kết quả: Mô hình lưới cho (3,0,đông). Mô hình bánh xe cho dịch chuyển lý tưởng 160 mm. Mỗi kết quả chỉ có nghĩa trong mô hình và đơn vị của nó.

Tách rõ mô hình giúp bạn dùng bàn lưới để học thuật toán và dùng phép đo/cấu hình để kiểm tra cơ cấu thật. Một chương trình đúng thứ tự vẫn cần phần cứng hoạt động và được cấu hình đúng.

Tự giải thích rồi đối chiếu

Nếu chỉ biết robot ở (3,0), có đoán được F tiếp theo không?

Chưa. Nếu hướng đông thì x tăng; nếu hướng nam thì y tăng. Cần biết hướng cùng với vị trí.

Hai lệnh motor cùng số dương có luôn làm robot tiến không?

Không thể kết luận nếu chưa biết cách lắp và cấu hình chiều quay. Hai bánh phải tạo vận tốc tiếp tuyến cùng đẩy robot tiến theo quy ước đã chọn.

Chọn độ sâu phù hợp

Mới bắt đầu · Đóng vai robot bằng mũi tên giấy

Đặt một mũi tên tại (0,0), hướng đông. Di chuyển đúng một ô cho từng F và nói vị trí mới; chưa cần đọc Python hoặc cấu hình động cơ.

Dự đoán và ghi ba trạng thái trước khi chạy lại F,F,F trong Lab.

Đã học cơ bản · Nối lệnh, cơ cấu và phép đo

Hãy phân biệt "yêu cầu đi" với "đã đi". Từ vận tốc và thời gian có thể ước lượng quãng đường, nhưng kiểm chứng thực cần thước hoặc dữ liệu phù hợp.

Nêu ba yếu tố phải biết trước khi chuyển một nhiệm vụ lưới sang VEX IQ thật.

Muốn tìm hiểu sâu · Xét mô hình truyền động vi sai

v = (vL + vR)/2 mô tả vận tốc tâm trong mô hình lăn không trượt. Khi hai bánh khác vận tốc, hướng robot cũng thay đổi, nên chỉ tính trung bình vận tốc chưa đủ để xác định đường đi. Ma sát, độ rơ và mặt sân có thể làm kết quả thực lệch mô hình.

Giải thích trường hợp vL = −vR: vì sao tâm có vận tốc tiến bằng 0 nhưng robot vẫn có thể đang quay?

Kiến thức trọng tâm

Việc đầu tiên: robot làm theo thứ tự

Một chương trình là danh sách yêu cầu robot thực hiện. Trên bàn lưới, F đưa robot tiến một ô theo hướng đang nhìn; R và L đổi hướng tại chỗ. Bắt đầu bằng F,F,F để dễ theo dõi. Trong robot VEX IQ, Brain là bộ điều khiển chạy chương trình, còn động cơ và bánh xe tạo chuyển động. Python/GearsBot là hoạt động mở rộng sau khi đã hiểu chuỗi lệnh.

Đọc thêm · Ba phần của hệ điều khiển

Brain lưu và thực thi chương trình. Cảm biến biến đại lượng như khoảng cách hoặc góc thành dữ liệu đầu vào. Động cơ nhận lệnh để tạo chuyển động; bánh xe và khung truyền chuyển động ra môi trường. Chương trình không tự biết robot đã đi bao xa nếu không có số đo hoặc mô hình phù hợp.

Đọc thêm · Robot hai bánh điều khiển vi sai

Xét vận tốc tiếp tuyến của bánh trái vL và bánh phải vR, cùng quy ước dương là đẩy robot tiến. Trong mô hình lăn không trượt, vận tốc tâm v = (vL + vR)/2. Khi vL = vR, robot lý tưởng đi thẳng. Hai động cơ lắp đối xứng có thể cần đảo chiều cấu hình để cùng tạo chuyển động tiến; không suy chiều tiến chỉ từ dấu lệnh motor.

Đọc thêm · Lưới và phần cứng là hai mô hình khác nhau

Trong bàn thí nghiệm, x tăng sang phải, y tăng xuống, hướng đầu là đông; một lệnh tiến đổi đúng một ô. Ô lưới không tự tương ứng với 100 mm trên VEX IQ. Với robot thật phải cấu hình Drivetrain theo cổng, bánh xe và cơ cấu thực tế, rồi đo để hiệu chuẩn. VEXcode IQ dành cho IQ; GearsBot chỉ bổ trợ thuật toán trên robot của chính nền tảng đó.

Ví dụ có lời giải

Nếu cả hai bánh có vận tốc tiếp tuyến 100 mm/s trong 2 s, mô hình lý tưởng cho s = v × t = 200 mm. Trên lưới, từ (0,0) hướng đông, ba lệnh tiến cho (3,0); không được lấy kết quả này làm bằng chứng robot thật đã đi 300 mm.

Lỗi thường gặp và cách tránh

1. Hai motor cùng quay theo chiều kim đồng hồ nhìn từ hai phía không có nghĩa cả hai bánh đang đẩy robot tiến.

2. Một chuỗi lệnh đúng trong mô hình lưới chưa chứng minh robot thật đi đúng quãng đường hoặc không trượt.

Các bước thực hành

1. Dự đoán: vẽ ba ô trước mặt robot, viết đích của F,F,F từ vị trí (0,0), đầu quay sang phải (hướng đông). Quy ước x tăng sang phải, y tăng xuống.

2. Thao tác: vào Lab → Bàn thí nghiệm, đặt lại robot và chạy F,F,F. Đây là chuỗi lệnh của mô hình lưới.

3. Quan sát: ghi vị trí và hướng cuối; kiểm tra robot đã tiến ba ô nhưng vẫn nhìn cùng hướng. So sánh với hình vẽ trước khi chạy.

4. Giải thích: vẽ chương trình → Brain (bộ điều khiển) → động cơ → bánh xe; thêm cảm biến → Brain. Chỉ phần nào ra lệnh và phần nào tạo chuyển động.

5. Thử tiếp: dự đoán F,F,F,R,F,F, đặt lại rồi chạy. Ghi riêng thời điểm rẽ: vị trí giữ nguyên, hướng thay đổi.

6. Mở rộng: nếu dùng mã Python dài trong Lab, chọn đúng robot GearsBot theo hướng dẫn và thử chạy thẳng 200 mm. Ghi kết quả ở bảng riêng; số ô trên lưới chưa đo quãng đường của robot VEX IQ thật.

Mã mẫu · Python · GearsBot (ev3dev2)

```
from math import pi, sqrt
from time import time, sleep
from ev3dev2.motor import MoveTank, OUTPUT_A, OUTPUT_B
from ev3dev2.sensor import INPUT_4
from ev3dev2.sensor.virtual import GPSSensor

tank = MoveTank(OUTPUT_A, OUTPUT_B)
gps = GPSSensor(INPUT_4)
WHEEL_DIAMETER_MM = 56.0
TRACK_WIDTH_MM = 152.0
SPEED = 25

def move_wheels(left_speed, right_speed, degrees, timeout_s=8.0):
  start_left = tank.left_motor.position
  start_right = tank.right_motor.position
  deadline = time() + timeout_s
  tank.on_for_degrees(left_speed, right_speed, degrees,
              brake=True, block=False)
  try:
    sleep(0.05)
    while time() < deadline:
      if tank.left_motor.is_stalled or tank.right_motor.is_stalled:
        raise RuntimeError("Motor stalled; check the course")
      if not (tank.left_motor.is_running or tank.right_motor.is_running):
        left_done = abs(tank.left_motor.position - start_left)
        right_done = abs(tank.right_motor.position - start_right)
        if min(left_done, right_done) >= degrees - 5:
          return
      sleep(0.02)
    raise RuntimeError("Motion timeout; reset before retrying")
  finally:
    tank.off(brake=True)

def forward(distance_mm):
  degrees = 360.0 * distance_mm / (pi * WHEEL_DIAMETER_MM)
  move_wheels(SPEED, SPEED, degrees)

def turn_right(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(SPEED, -SPEED, degrees)

def turn_left(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(-SPEED, SPEED, degrees)

start = gps.position
try:
  forward(200)
  end = gps.position
  displacement_mm = 10 * sqrt((end[0] - start[0]) ** 2
                    + (end[1] - start[1]) ** 2)
  print("Requested distance (mm):", 200)
  print("GPS displacement (mm):", round(displacement_mm, 1))
  print("Wheel angles (deg):", tank.left_motor.position,
      tank.right_motor.position)
finally:
  tank.off(brake=True)
```

Chọn và sử dụng phòng lab

Trong GearsBot, chọn robot Single Sensor Line Follower (singleFollower), mở Python và dán toàn bộ chương trình. Robot dùng bánh trái A, bánh phải B; đường kính bánh 56 mm, khoảng cách hai bánh 152 mm. Chọn sân rộng, phẳng; Reset trước mỗi lượt rồi Run. Chương trình chạy thẳng 200 mm ở tốc độ 25%, ghi độ dời GPS và góc motor. GPS của GearsBot trả về cm nên code nhân 10 để đổi sang mm. So sánh yêu cầu với kết quả đo, lặp ba lượt để quan sát sai số; đây là phép thử trên robot GearsBot.

Bài tập áp dụng

Từ (0,0,đông), dự đoán đích của F,F,F,R,F,F bằng một hình mũi tên hoặc bảng (x,y,hướng). Đặt lại, chạy chuỗi trên bàn lưới và đối chiếu sau ba lệnh tiến, sau lần rẽ và tại đích. Viết một câu giải thích vai trò của Brain; phần thêm: nêu điều phải đo trước khi đổi ô thành mm.

Gợi ý đối chiếu sau khi tự làm

F,F,F → (3,0,đông); R → (3,0,nam); F,F → (3,2,nam). Brain thực thi chương trình và gửi lệnh đến động cơ; bánh xe truyền chuyển động ra mặt sân. Muốn đổi ô thành mm phải định nghĩa tỷ lệ và đo/hiệu chuẩn robot; lưới không tự xác nhận quãng đường thật.

Tự kiểm tra mức hiểu

1. Sau F,F,F, vị trí nào đổi và hướng có đổi không?

2. Khi robot quay phải tại chỗ, vì sao chưa cộng thêm một ô đã đi?

Câu hỏi trắc nghiệm

Bộ phận nào thực thi chương trình điều khiển?

1. Bánh xe

2. Brain

3. Khung robot

Đáp án: 2. Brain là bộ điều khiển thực thi chương trình. Motor tạo chuyển động và bánh xe truyền lực; chúng không thay vai trò xử lý chương trình của Brain.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEX IQ STEM Labs — bài học và quy trình kỹ thuật

https://education.vex.com/stemlabs/iq

VEXcode IQ thế hệ 2 — tài liệu Python chính thức

https://api.vex.com/iq2/home/python/

GEARS — cấu hình Single Sensor Line Follower

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GEARS — API ev3dev2.motor trong simulator

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GEARS — API GPS và đơn vị tọa độ

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/virtual.py

Bài 2/6 · Góc quay và đường đi

Theo dõi đồng thời vị trí và hướng khi robot tiến, rẽ trái hoặc rẽ phải.

Chuẩn bị

Hoàn thành bài Cấu trúc và truyền động; biết một vòng là 360°, góc vuông là 90° và quy ước tọa độ của bàn lưới.

Sau bài học, bạn có thể

1. Theo dõi đồng thời vị trí và hướng khi robot tiến, rẽ trái hoặc rẽ phải.

2. Giải thích vì sao góc quay thân robot khác góc quay của từng bánh.

3. Đo và báo cáo sai số góc qua nhiều lần thử với cùng điều kiện.

Cùng hiểu bài từ đầu

Robot rẽ làm thay đổi hướng; bánh xe quay làm thay đổi góc bánh. Hai góc này không cùng một đại lượng. Bạn sẽ đi từ việc xoay một mũi tên trên lưới tới giải thích vì sao thân robot quay 90° có thể cần bánh quay nhiều hơn 90°.

Đặt một thước ngang lên bàn và quay thước quanh trung điểm. Hai đầu thước đi theo hai cung tròn ngược chiều, còn trung điểm vẫn ở chỗ cũ.

Khi robot hai bánh quay tại chỗ lý tưởng, mỗi bánh đi trên một cung quanh tâm robot. Bánh cách tâm càng xa thì phải đi cung càng dài để thân quay cùng một góc.

Giới hạn của ví von: Một thước cứng không thể hiện trượt bánh, đàn hồi hoặc sai số điều khiển. Phép tính hình học chỉ áp dụng gần đúng khi cơ cấu và điều kiện lăn đáp ứng giả định.

Những từ cần hiểu

1. Góc thân robot · độ: Mức đổi hướng của cả robot. Rẽ phải 90° biến hướng đông thành nam theo lưới của bài.

2. Góc bánh · độ hoặc vòng: Mức bánh quay quanh chính trục của nó; không đồng nhất với góc thân.

3. Bề rộng vệt bánh · b: Khoảng cách giữa hai đường tiếp xúc của bánh trái và phải; mỗi bánh cách tâm khoảng b/2 trong mô hình đối xứng.

4. Radian · rad: Đơn vị góc dùng trong s = rθ. 180° = π rad; phải đổi độ sang rad khi dùng công thức này.

Thân quay 90°, mỗi bánh cần quay bao nhiêu?

Bắt đầu với: Giả sử b = 140 mm, đường kính bánh D = 70 mm, quay tại chỗ 90° và lăn không trượt.

Tìm bán kính quỹ đạo của bánh

Tâm quay ở giữa hai bánh nên mỗi bánh đi theo cung bán kính b/2 = 70 mm. Đổi góc thân: 90° = π/2 rad.

Tính chiều dài cung mỗi bánh

sB = 70 × π/2 = 35π mm, khoảng 109,96 mm. Hai bánh đi cùng độ dài cung nhưng theo hai chiều ngược nhau để quay tại chỗ.

Đổi cung đi thành góc bánh

Chu vi bánh là πD = 70π mm. Mỗi bánh lăn 35π/(70π) = 0,5 vòng, tức 180°. Đó là góc bánh, trong khi góc thân chỉ 90°.

Kết quả: Trong mô hình này, hai bánh quay ngược chiều khoảng 180° mỗi bánh để thân quay 90°. Vị trí tâm được giữ, hướng thay đổi.

Góc yêu cầu còn phải được thử và đo trên robot thật. Không lấy kết quả hình học làm bằng chứng đã bù hết trượt bánh hay độ rơ cơ khí.

Tự giải thích rồi đối chiếu

Rẽ tại chỗ trên lưới có tự làm tăng x hoặc y không?

Không. Lệnh rẽ chỉ đổi hướng trong mô hình lưới. Lệnh F sau đó mới đổi vị trí theo hướng mới.

Nếu tăng b mà giữ D và góc thân, bánh cần quay nhiều hơn hay ít hơn?

Nhiều hơn. Bánh ở xa tâm hơn phải đi cung dài hơn; cùng chu vi bánh thì cần nhiều vòng hơn.

Chọn độ sâu phù hợp

Mới bắt đầu · Xoay mũi tên, giữ nguyên ô

Tập rẽ đông → nam → tây → bắc. Đánh dấu ô đang đứng để thấy việc xoay hướng khác với tiến sang ô khác.

Từ hướng bắc, dự đoán hướng sau R rồi F sẽ làm tọa độ nào thay đổi.

Đã học cơ bản · Dùng hình học trước phép chia

Vẽ tâm quay và hai đường bánh, ghi b/2 trên bán kính. Sau khi tính cung, mới chia cho chu vi bánh để ra số vòng.

Tính lại trường hợp thân quay 180° với cùng b và D.

Muốn tìm hiểu sâu · Đo sai số quay có dấu

Cố định mặt sân, tốc độ, pin và cách đo góc; ghi từng lượt thay vì chỉ một kết quả đẹp. Nếu thân thường quay thiếu, tìm tham số mô hình hoặc cấu hình phù hợp rồi thay một yếu tố mỗi lần. Công thức lý tưởng không giải quyết sai số đổi theo tải và tốc độ.

Lập bảng yêu cầu 45°, 90°, 180° và góc thực đo; kiểm tra giả thuyết sai số tỷ lệ trước khi chọn một hệ số bù chung.

Kiến thức trọng tâm

Trạng thái phải có cả vị trí lẫn hướng

Một trạng thái có thể ghi (x,y,hướng). Tiến một ô phụ thuộc hướng hiện tại: đông tăng x, nam tăng y, tây giảm x, bắc giảm y. Rẽ tại chỗ 90° giữ nguyên vị trí lý tưởng và chỉ đổi hướng. Vì vậy, cùng số lệnh tiến nhưng xen các lệnh rẽ khác nhau có thể dẫn tới các đích khác nhau.

Góc thân robot và chuyển động bánh

Với hai bánh quay ngược chiều và tâm robot đứng yên, mỗi bánh đi một cung sB = b × |θ|/2. b là khoảng cách giữa hai đường tiếp xúc bánh; θ là góc thân tính bằng radian, với θrad = θđộ × π/180. Nếu bánh đường kính D lăn không trượt, góc quay mỗi bánh tính bằng độ là 360 × sB/(πD). Đây là mô hình lý tưởng, không tự bù trượt hoặc độ rơ.

Hiệu chuẩn bằng đo lặp

Lệnh rẽ 90° là giá trị yêu cầu, không phải bảo đảm góc thực tế đúng 90°. Đánh dấu hướng đầu trên sàn, đo góc cuối ở cùng tốc độ, cùng mặt sàn và cùng pin. Ghi từng lượt và độ lệch có dấu; chỉ thay một tham số mỗi lần để biết nguyên nhân cải thiện.

Ví dụ có lời giải

Giả sử b = 120 mm, D = 60 mm, thân robot quay 90° = π/2 rad. Mỗi bánh cần đi sB = 120 × π/4 ≈ 94,25 mm; góc bánh ≈ 180°. Như vậy, thân quay 90° không có nghĩa mỗi bánh quay 90°.

Lỗi thường gặp và cách tránh

1. Góc bánh quay không bằng góc thân robot quay; phải xét đường kính bánh và khoảng cách giữa hai bánh.

2. Quay tổng cộng 360° không bảo đảm robot thật về đúng hướng đầu nếu từng lượt bị trượt hoặc sai số.

Các bước thực hành

1. Trên giấy, lập bảng cho F,R,F từ (0,0,đông). Phân biệt hàng đổi vị trí và hàng chỉ đổi hướng.

2. Đặt lại bàn lưới; chạy từng lệnh rồi kiểm tra lần lượt (1,0,đông), (1,0,nam), (1,1,nam).

3. Dự đoán kết quả của bốn lần rẽ phải liên tiếp và của một đường hình vuông. Chạy và kiểm tra cả vị trí lẫn hướng.

4. Nếu có kit: đo ba lượt quay 90° ở tốc độ thấp; ghi sai số từng lượt, không lấy một lượt đẹp nhất làm kết luận.

Mã mẫu · Python · GearsBot (ev3dev2)

```
from math import pi, sqrt
from time import time, sleep
from ev3dev2.motor import MoveTank, OUTPUT_A, OUTPUT_B
from ev3dev2.sensor import INPUT_4
from ev3dev2.sensor.virtual import GPSSensor

tank = MoveTank(OUTPUT_A, OUTPUT_B)
gps = GPSSensor(INPUT_4)
WHEEL_DIAMETER_MM = 56.0
TRACK_WIDTH_MM = 152.0
SPEED = 25

def move_wheels(left_speed, right_speed, degrees, timeout_s=8.0):
  start_left = tank.left_motor.position
  start_right = tank.right_motor.position
  deadline = time() + timeout_s
  tank.on_for_degrees(left_speed, right_speed, degrees,
              brake=True, block=False)
  try:
    sleep(0.05)
    while time() < deadline:
      if tank.left_motor.is_stalled or tank.right_motor.is_stalled:
        raise RuntimeError("Motor stalled; check the course")
      if not (tank.left_motor.is_running or tank.right_motor.is_running):
        left_done = abs(tank.left_motor.position - start_left)
        right_done = abs(tank.right_motor.position - start_right)
        if min(left_done, right_done) >= degrees - 5:
          return
      sleep(0.02)
    raise RuntimeError("Motion timeout; reset before retrying")
  finally:
    tank.off(brake=True)

def forward(distance_mm):
  degrees = 360.0 * distance_mm / (pi * WHEEL_DIAMETER_MM)
  move_wheels(SPEED, SPEED, degrees)

def turn_right(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(SPEED, -SPEED, degrees)

def turn_left(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(-SPEED, SPEED, degrees)

try:
  forward(200)
  turn_right(90)
  forward(200)
  print("Commands: forward 200 mm, right 90 deg, forward 200 mm")
  print("GPS endpoint (cm):", gps.position[:2])
  print("Right-turn wheel target (deg):",
      round(90 * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM, 1))
finally:
  tank.off(brake=True)
```

Chọn và sử dụng phòng lab

Trong GearsBot, chọn robot Single Sensor Line Follower (singleFollower), mở Python và dán toàn bộ chương trình. Robot dùng bánh trái A, bánh phải B; đường kính bánh 56 mm, khoảng cách hai bánh 152 mm. Chọn sân rộng, phẳng; Reset trước mỗi lượt rồi Run. Code thực hiện tiến 200 mm, quay phải tại chỗ 90° rồi tiến 200 mm. Góc quay mỗi bánh được tính từ kích thước robot, khác góc quay thân. Đọc vị trí cuối ở console; giữ cùng sân và điểm đầu khi đo góc thực tế bằng công cụ đo của sân.

Bài tập áp dụng

Robot đo được các góc 88°, 91°, 89° khi yêu cầu 90°. Tính sai số có dấu của mỗi lượt và sai số trung bình. Giải thích vì sao trung bình nhỏ chưa có nghĩa từng lượt đều chính xác.

Gợi ý đối chiếu sau khi tự làm

Sai số đo - mục tiêu lần lượt −2°, +1°, −1°; trung bình −2/3° ≈ −0,67°. Sai số trung bình có thể triệt tiêu một phần giữa dương và âm; vẫn phải báo từng lượt hoặc mức phân tán. Trong lưới lý tưởng, bốn rẽ 90° về hướng ban đầu; robot thật có thể còn sai lệch.

Tự kiểm tra mức hiểu

1. Viết được đủ bốn trạng thái của chuỗi F,R,F kể cả trạng thái đầu.

2. Đổi được 90° thành π/2 rad trước khi dùng công thức cung tròn.

3. Báo cáo được ba phép đo và giải thích hạn chế của chỉ số trung bình.

Câu hỏi trắc nghiệm

Sau tiến–phải–tiến, robot từ (0,0) hướng đông tới…

1. (2,0)

2. (1,1)

3. (0,2)

Đáp án: 2. Tiến đầu tiên tăng x từ 0 lên 1. Rẽ phải đổi hướng đông thành nam nhưng giữ nguyên vị trí. Tiến cuối tăng y lên 1, nên đích là (1,1).

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEX IQ STEM Labs — bài học và quy trình kỹ thuật

https://education.vex.com/stemlabs/iq

VEXcode IQ thế hệ 2 — tài liệu Python chính thức

https://api.vex.com/iq2/home/python/

GEARS — cấu hình Single Sensor Line Follower

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GEARS — API ev3dev2.motor trong simulator

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GEARS — API GPS và đơn vị tọa độ

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/virtual.py

Bài 3/6 · Dừng theo cảm biến

Viết điều kiện dừng dựa trên khoảng cách với ngưỡng và đơn vị rõ ràng.

Chuẩn bị

Biết câu điều kiện nếu–ngược lại và so sánh <, ≤; phân biệt cm và mm. Hoàn thành bài chuyển động và góc quay.

Sau bài học, bạn có thể

1. Viết điều kiện dừng dựa trên khoảng cách với ngưỡng và đơn vị rõ ràng.

2. Kiểm thử ba trường hợp dưới, bằng và trên ngưỡng cùng trường hợp dữ liệu không hợp lệ.

3. Giải thích độ trễ phản ứng và vì sao phải giữ khoảng dự phòng khi chạy robot thật.

Cùng hiểu bài từ đầu

Dừng theo cảm biến cần hơn một câu "gặp vật thì dừng". Bạn cần khoảng cách hợp lệ, một ranh giới rõ ràng và thời gian để robot đọc rồi hãm. Bài này tách logic quyết định khỏi quãng đường robot còn đi sau khi cần dừng.

Khi đạp xe, bạn không đứng yên ngay lúc nhìn thấy vật. Bạn cần thời gian phản ứng rồi thêm một đoạn đường để phanh.

Robot cũng có độ trễ giữa hai lượt đo và thời gian xử lý; sau lệnh dừng, cơ cấu còn cần hãm. Vì vậy tốc độ lớn hơn thường đòi hỏi khoảng dự phòng lớn hơn.

Giới hạn của ví von: Khả năng phanh của xe đạp và robot khác nhau. Công thức trong bài giả sử hãm đều và chỉ là ước lượng; còn phải đo thực tế, xét trượt và sai số cảm biến.

Những từ cần hiểu

1. Ngưỡng dừng · cm hoặc mm: Ranh giới so sánh với khoảng cách. Cả số đo và ngưỡng phải dùng cùng đơn vị.

2. Độ trễ · τ, s: Thời gian robot tiếp tục đi trước khi hiệu lực hãm bắt đầu, gồm các thành phần cần xét trong mô hình.

3. Gia tốc hãm · a, m/s²: Độ lớn tốc độ giảm theo thời gian trong giả định hãm đều; dùng $a > 0$ khi tính khoảng hãm.

4. Dữ liệu hợp lệ: Số đo có trạng thái và phạm vi phù hợp theo thiết bị. Không có dữ liệu hợp lệ phải xử lý riêng, không coi là đường trống.

Quyết định dừng và quãng đường dừng là hai phép kiểm tra

Bắt đầu với: Logic dùng T = 20 cm, d < T thì dừng. Phép tính động học riêng giả sử v = 0,30 m/s, τ = 0,10 s và a = 0,60 m/s².

Thử ba đầu vào quanh ngưỡng

19 cm chọn dừng; 20 cm và 21 cm chọn tiến chậm vì bài dùng dấu <. Nếu nhận dữ liệu lỗi, chọn dừng và báo lỗi thay vì so một khoảng cách giả.

Tính quãng đi trong độ trễ

$v\tau = 0{,}30 \times 0{,}10 = 0{,}030$ m. Robot đã đi thêm 3 cm trong thời gian chưa bắt đầu hãm của mô hình.

Tính và cộng khoảng hãm

$v^2/(2a) = 0{,}30^2/(2 \times 0{,}60) = 0{,}075$ m. Tổng dự kiến là 0,030 + 0,075 = 0,105 m = 10,5 cm, trước khi cộng các dự phòng khác.

Kết quả: Logic có ranh giới rõ; mô hình động học cho thêm 10,5 cm từ độ trễ và hãm theo các giả định. Hai kết quả không thay thế nhau.

Chọn ngưỡng thực phải xét khoảng dừng cùng kích thước robot, vị trí cảm biến, sai số và điều kiện thử. Chỉ việc chương trình gọi lệnh dừng chưa chứng minh không va chạm.

Tự giải thích rồi đối chiếu

Khoảng cách 200 mm so với ngưỡng 20 cm có nhỏ hơn không?

Không. 200 mm = 20 cm, đúng bằng ngưỡng. Phải đổi về cùng đơn vị trước khi so sánh.

Nếu tăng gấp đôi v, thành phần $v^2/(2a)$ tăng mấy lần khi a giữ nguyên?

Tăng bốn lần vì v được bình phương. Thành phần trễ $v\tau$ chỉ tăng hai lần nếu τ giữ nguyên.

Chọn độ sâu phù hợp

Mới bắt đầu · Bắt đầu bằng bảng quyết định

Dùng bốn thẻ: 19 cm, 20 cm, 21 cm và “lỗi”. Ghép mỗi thẻ với dừng/tiến và lý do. Chưa cần dùng công thức hãm để hiểu điều kiện.

Viết ra điều robot phải làm nếu mất số đo giữa lúc đang tiến.

Đã học cơ bản · Nối tốc độ với thời gian phản ứng

Đổi mọi đại lượng về m, s trước phép tính động học. Tách khoảng trễ và khoảng hãm để biết việc giảm tốc hoặc tăng tần suất đo ảnh hưởng phần nào.

So sánh tổng ước lượng khi τ giảm còn 0,05 s, giữ các thông số khác.

Muốn tìm hiểu sâu · Kiểm chứng mô hình dừng bằng dữ liệu

Đo nhiều lượt ở từng tốc độ và điều kiện mặt sân. Hãm đều là giả định đơn giản; tải, pin và độ bám có thể làm a thay đổi. Xác nhận cách thiết bị báo lỗi theo đúng thế hệ VEX IQ; nút dừng thủ công phải sử dụng được khi thử phần cứng.

Thiết kế bảng ghi tốc độ, độ trễ ước lượng, khoảng dừng đo được và điều kiện sân để đánh giá phạm vi mô hình còn phù hợp.

Kiến thức trọng tâm

Từ số đo đến hành động

Quy tắc của bài là: nếu $d < T$ thì dừng, ngược lại tiến chậm. d là khoảng cách đo và T là ngưỡng, phải cùng đơn vị. Với dấu <, trường hợp $d = T$ đi vào nhánh ngược lại. Dùng dấu $\leq$ là một yêu cầu khác và cần đổi bảng kiểm thử. Không trộn giá trị cm với ngưỡng mm.

Khoảng cách dừng có độ trễ

Robot còn di chuyển giữa hai lần đọc cảm biến và trong lúc hãm. Mô hình đơn giản có quãng đường dự phòng $s \approx v\tau + v^2/(2a)$, với v theo m/s, τ là tổng độ trễ theo s và $a > 0$ là độ lớn gia tốc hãm theo m/s². Công thức giả định hãm đều; thực tế phải đo thêm sai số cảm biến và độ trượt, không lấy kết quả tính làm khoảng an toàn tuyệt đối.

Dữ liệu lỗi và quyền dừng

Cảm biến khác thế hệ VEX IQ có khả năng và cách báo kết quả khác nhau. Kiểm tra thiết bị đang cấu hình và tài liệu đúng phiên bản. Không có số đo hợp lệ không có nghĩa phía trước trống: bài này yêu cầu dừng, báo lỗi và chờ kiểm tra. Nút dừng thủ công phải luôn sử dụng được khi thử phần cứng.

Ví dụ có lời giải

$T = 20$ cm và điều kiện $d < T$: 19 cm → dừng; 20 cm → tiến chậm; 21 cm → tiến chậm. Với $v = 0{,}20$ m/s, $\tau = 0{,}10$ s, $a = 0{,}50$ m/s², mô hình cho $s = 0{,}02 + 0{,}04 = 0{,}06$ m = 6 cm trước khi cộng dự phòng sai số.

Lỗi thường gặp và cách tránh

1. Giá trị cảm biến không hợp lệ không được coi như khoảng cách vô hạn để tiếp tục tiến.

2. Phát lệnh dừng không khiến robot thật đứng yên tức thời; tăng tốc đòi hỏi kiểm tra lại khoảng dự phòng.

Các bước thực hành

1. Viết rõ điều kiện < và đơn vị cm. Lập bảng dự đoán cho d = 19, 20, 21 cm khi T = 20 cm.

2. Trong bàn cảm biến, nhập từng cặp d,T và đối chiếu nhánh. Giữ cùng d = 25 cm, đổi T từ 20 lên 30 cm để thấy vai trò ngưỡng.

3. Thêm một hàng dữ liệu không hợp lệ trong bảng thiết kế và nêu phản ứng dừng/báo lỗi; mini lab số học không thay thế mô phỏng lỗi cảm biến thật.

4. Nếu có kit: chạy chậm trên sân trống, tăng khoảng dự phòng trước khi tăng tốc. Ghi quãng đường từ lúc phát lệnh dừng tới khi robot đứng yên.

Mã mẫu · Python · GearsBot (ev3dev2)

```
from time import time, sleep
from ev3dev2.motor import MoveTank, OUTPUT_A, OUTPUT_B
from ev3dev2.sensor import INPUT_2
from ev3dev2.sensor.lego import UltrasonicSensor

tank = MoveTank(OUTPUT_A, OUTPUT_B)
distance_sensor = UltrasonicSensor(INPUT_2)
THRESHOLD_CM = 20.0
TIME_LIMIT_S = 10.0
reason = "time_limit"
last_distance_cm = None

def read_distance_cm():
  value = distance_sensor.distance_centimeters
  if not 0 < value <= 255:
    raise ValueError("Invalid ultrasonic reading")
  return value

try:
  deadline = time() + TIME_LIMIT_S
  while time() < deadline:
    last_distance_cm = read_distance_cm()
    if last_distance_cm < THRESHOLD_CM:
      reason = "obstacle_threshold"
      break
    tank.on(15, 15)
    sleep(0.05)
finally:
  tank.off(brake=True)

print("Stop reason:", reason)
print("Last distance (cm):", last_distance_cm)
```

Chọn và sử dụng phòng lab

Trong GearsBot, chọn robot Single Sensor Line Follower (singleFollower), mở Python và dán toàn bộ chương trình. Robot dùng bánh trái A, bánh phải B; đường kính bánh 56 mm, khoảng cách hai bánh 152 mm. Chọn sân rộng, phẳng; Reset trước mỗi lượt rồi Run. Cảm biến siêu âm nằm ở INPUT_2. Đặt một hộp trước robot trên sân, chạy và đọc khoảng cách dừng ở console. Ngưỡng là 20 cm với dấu <; đổi thành 30 để làm bài tập. Giá trị 255 cm là hết tầm đo mặc định. Chương trình dừng sau tối đa 10 giây hoặc khi đọc lỗi; tốc độ, độ trễ và quãng hãm vẫn ảnh hưởng khoảng cách dừng thực tế.

Bài tập áp dụng

Dùng T = 30 cm: dự đoán d = 29, 30, 31 cm với dấu <. Đổi d từ cm sang mm rồi tính lại để kiểm tra tính nhất quán. Nêu điều gì cần thay đổi nếu muốn robot dừng cả khi d = T.

Gợi ý đối chiếu sau khi tự làm

29 cm dừng; 30 và 31 cm tiến theo quy tắc <. Khi dùng mm: d = 290, 300, 310 mm, T = 300 mm cho kết quả tương tự. Muốn dừng cả khi bằng ngưỡng, dùng d ≤ T và cập nhật ca biên. Trên robot thật cần tính/đo độ trễ và quãng đường hãm.

Tự kiểm tra mức hiểu

1. Lập đúng bảng dưới/bằng/trên ngưỡng theo dấu <.

2. Đổi được 20 cm thành 200 mm và giữ thống nhất đơn vị so sánh.

3. Nêu được phản ứng với dữ liệu lỗi và ít nhất hai thành phần của khoảng dừng thực tế.

Câu hỏi trắc nghiệm

Với 25 < 30, robot thực hiện nhánh nào?

1. Dừng

2. Tiến

3. Rẽ trái

Đáp án: 1. 25 < 30 là đúng nên thực hiện nhánh dừng. Đáp án này phụ thuộc điều kiện đã nêu; ở đúng ngưỡng, dấu < và ≤ cho hai hành vi khác nhau.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEX IQ STEM Labs — bài học và quy trình kỹ thuật

https://education.vex.com/stemlabs/iq

VEXcode IQ thế hệ 2 — tài liệu Python chính thức

https://api.vex.com/iq2/home/python/

GEARS — cấu hình Single Sensor Line Follower

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GEARS — API ev3dev2.motor trong simulator

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GEARS — API cảm biến siêu âm

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/lego.py

Bài 4/6 · Nhiệm vụ sa bàn

Chuyển một nhiệm vụ sa bàn thành điều kiện đầu, đích và tiêu chí thành công có thể kiểm tra.

Chuẩn bị

Hoàn thành ba bài chuyển động, góc quay và cảm biến. Biết ghi bảng (lần thử, chương trình, điều kiện, kết quả).

Sau bài học, bạn có thể

1. Chuyển một nhiệm vụ sa bàn thành điều kiện đầu, đích và tiêu chí thành công có thể kiểm tra.

2. Thiết kế, thử và so sánh hai chuỗi lệnh trong cùng điều kiện.

3. Ghi cả lượt thất bại và giới hạn của kết quả thử nghiệm.

Cùng hiểu bài từ đầu

Một nhiệm vụ robot chỉ kiểm tra được khi bạn biết thế nào là hoàn thành. Hãy xác định điểm đầu, hướng đầu, đích và giới hạn trước, rồi chia đường đi thành các đoạn có thể đối chiếu. Cách làm này giúp tìm lệnh gây sai ngay tại chỗ.

Chỉ đường cho một người tới lớp học cần biết họ đang đứng đâu và quay về hướng nào. “Đi ba bước rồi rẽ phải” sẽ đưa tới nơi khác nếu người ấy bắt đầu quay ngược.

Điểm đầu và hướng đầu là dữ liệu của nhiệm vụ. Các mốc giữa đường giống điểm kiểm tra: nếu sai ở một mốc, kiểm tra đoạn vừa đi thay vì đổi toàn bộ chỉ dẫn.

Giới hạn của ví von: Bước chân và ô lưới chỉ hỗ trợ lập kế hoạch. Robot thật cần đơn vị quãng đường, cấu hình và số đo phù hợp; không thể chuyển nguyên số ô thành mm tùy ý.

Những từ cần hiểu

1. Đặc tả nhiệm vụ: Danh sách điều kiện đầu vào, điều cần đạt và giới hạn. Nó cho phép hai người đánh giá cùng một kết quả.

2. Mốc kiểm tra: Trạng thái dự kiến ở giữa đường; dùng để khoanh vùng nơi chương trình bắt đầu lệch kế hoạch.

3. Ràng buộc: Điều không được vi phạm, như không ra ngoài lưới hoặc không đi qua ô cấm.

4. Tiêu chí tối ưu: Đại lượng muốn giảm hoặc tăng sau khi đã hoàn thành đúng, ví dụ số rẽ hoặc thời gian.

Chia một tuyến giao hàng thành ba đoạn

Bắt đầu với: Lưới không vật cản; x sang phải, y xuống dưới. Xuất phát (0,0,đông), đích (2,1). Tuyến đề xuất F,F,R,F; không được vượt biên của bàn.

Đặt mốc sau đoạn thẳng đầu

Hai F từ hướng đông đưa robot lần lượt tới (1,0,đông) và (2,0,đông). Mốc cần đạt là (2,0,đông).

Kiểm tra hướng sau lệnh rẽ

R đưa hướng đông thành nam nhưng giữ vị trí (2,0). Nếu lúc này robot đã sang ô khác thì cách hiểu hoặc thao tác lệnh đang sai.

Đi đoạn cuối và xét hoàn thành

F khi hướng nam đưa tới (2,1,nam). Kiểm tra đúng đích, không vượt biên và dừng theo yêu cầu; sau đó mới đếm chi phí tuyến.

Kết quả: Tuyến dùng 3 bước tiến và 1 lần rẽ, kết thúc ở (2,1,nam). Nếu đề không yêu cầu hướng cuối thì vị trí này đáp ứng phần đích; các ràng buộc còn lại vẫn phải được kiểm tra.

Ghi trạng thái giữa các đoạn làm bản chạy có thể giải thích. Khi so hai tuyến, hãy đặt lại cùng điểm và hướng đầu; nếu không, sự khác nhau có thể chỉ do điều kiện xuất phát.

Tự giải thích rồi đối chiếu

Đến đúng tọa độ nhưng đi xuyên ô cấm có hoàn thành không?

Không. Hoàn thành phải đáp ứng cả đích và ràng buộc đã nêu. Đi nhanh hoặc ít lệnh không bù cho việc vi phạm đề bài.

Vì sao chỉ ghi “xuất phát ở (0,0)” chưa đủ cho tuyến F,F,R,F?

Thiếu hướng đầu. F phụ thuộc hướng; cùng tọa độ nhưng khác hướng có thể đi tới ô khác ngay lệnh đầu.

Chọn độ sâu phù hợp

Mới bắt đầu · Dùng bảng từng lệnh

Vẽ cột lệnh, x, y và hướng. Mỗi lần chỉ sửa phần trạng thái mà lệnh thực sự tác động. Bảng ngắn giúp bạn không phải nhớ cả đường đi trong đầu.

Tự đi bằng mũi tên trên giấy, rồi đối chiếu với Lab sau mỗi đoạn.

Đã học cơ bản · Kiểm tra đúng trước, so sánh chi phí sau

Tách hai câu hỏi: tuyến có đạt yêu cầu không và trong các tuyến đạt, tuyến nào có chi phí tốt hơn. Dùng cùng quy ước đếm cho mọi tuyến.

Đề xuất một tuyến thứ hai cùng đích rồi ghi số bước tiến và số rẽ của cả hai.

Muốn tìm hiểu sâu · Từ mốc lưới sang mốc đo được

Trên robot thật, sai số từng đoạn có thể tích lũy. Cân nhắc điểm kiểm tra bằng cảm biến hoặc dấu mốc có thể quan sát thay vì chỉ chờ một khoảng thời gian. Một bộ ba lần chạy cùng mô hình xác định chưa đánh giá khả năng gặp sân khác.

Viết một bộ ba nhiệm vụ có vị trí/hướng đầu khác nhau và nêu đoạn nào của chương trình cần tham số hóa để tái sử dụng.

Kiến thức trọng tâm

Đặc tả trước khi lập trình

Một nhiệm vụ cần có điểm đầu, hướng đầu, đích, vật cản, vùng không được vào và giới hạn thời gian hoặc số lệnh. Ví dụ trên lưới: xuất phát (0,0,đông), đến (3,2), không ra ngoài lưới. Nếu bỏ hướng đầu, cùng một chuỗi lệnh có thể cho đích khác.

Chia nhiệm vụ thành các đoạn kiểm tra được

Tách đường đi thành những đoạn ngắn: tiến tới mốc, quay, tiến tới vùng đích, dừng. Với mô hình lưới, có thể xác minh trạng thái sau từng lệnh. Với robot thật, mỗi đoạn cần dựa trên đơn vị đo/cảm biến phù hợp; một ô ảo không tự tương ứng với quãng đường trên sân.

Cải tiến dựa trên dữ liệu

Kiểm tra đạt yêu cầu trước rồi mới tối ưu thời gian hoặc số lệnh. Khi so sánh A và B phải giữ cùng trạng thái đầu, bản đồ và quy ước đếm. Ghi mọi lượt, gồm vượt biên/không tới đích. Lặp ba lần trên mô hình xác định giúp phát hiện thao tác sai, nhưng không chứng minh thuật toán tổng quát cho mọi sân mới.

Ví dụ có lời giải

Tuyến A: F,F,F,R,F,F gồm 5 lệnh tiến và 1 rẽ, đến (3,2,nam). Một tuyến vòng tới cùng đích có thể dùng nhiều hành động hơn; chỉ được so sánh chi phí sau khi cả hai tuyến đều đáp ứng điều kiện thành công.

Lỗi thường gặp và cách tránh

1. Một lượt tới đích không chứng minh robot có khả năng lặp lại ở mọi điều kiện.

2. Thuật toán ít lệnh nhưng không hoàn thành nhiệm vụ không phải giải pháp tốt hơn.

Các bước thực hành

1. Ghi đặc tả: điểm đầu/hướng đầu, đích (3,2), biên lưới và tiêu chí dừng. Đánh số phiên bản chương trình.

2. Vẽ tuyến A và dự đoán trạng thái sau từng đoạn. Chạy từ Reset, ghi đích thực tế và số lệnh tiến/rẽ.

3. Thiết kế tuyến B có cùng đích, chạy với cùng điều kiện và cùng cách đếm. Ghi cả lượt sai, không sửa bảng sau khi biết kết quả.

4. Chọn một thay đổi, nêu dự đoán cải thiện rồi thử lại; nếu có kit thật hãy đo ba lượt ở tốc độ thấp trên sân an toàn.

Mã mẫu · Python · GearsBot (ev3dev2)

```
from math import pi, sqrt
from time import time, sleep
from ev3dev2.motor import MoveTank, OUTPUT_A, OUTPUT_B
from ev3dev2.sensor import INPUT_4
from ev3dev2.sensor.virtual import GPSSensor

tank = MoveTank(OUTPUT_A, OUTPUT_B)
gps = GPSSensor(INPUT_4)
WHEEL_DIAMETER_MM = 56.0
TRACK_WIDTH_MM = 152.0
SPEED = 25

def move_wheels(left_speed, right_speed, degrees, timeout_s=8.0):
  start_left = tank.left_motor.position
  start_right = tank.right_motor.position
  deadline = time() + timeout_s
  tank.on_for_degrees(left_speed, right_speed, degrees,
              brake=True, block=False)
  try:
    sleep(0.05)
    while time() < deadline:
      if tank.left_motor.is_stalled or tank.right_motor.is_stalled:
        raise RuntimeError("Motor stalled; check the course")
      if not (tank.left_motor.is_running or tank.right_motor.is_running):
        left_done = abs(tank.left_motor.position - start_left)
        right_done = abs(tank.right_motor.position - start_right)
        if min(left_done, right_done) >= degrees - 5:
          return
      sleep(0.02)
    raise RuntimeError("Motion timeout; reset before retrying")
  finally:
    tank.off(brake=True)

def forward(distance_mm):
  degrees = 360.0 * distance_mm / (pi * WHEEL_DIAMETER_MM)
  move_wheels(SPEED, SPEED, degrees)

def turn_right(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(SPEED, -SPEED, degrees)

def turn_left(angle_deg=90):
  degrees = angle_deg * TRACK_WIDTH_MM / WHEEL_DIAMETER_MM
  move_wheels(-SPEED, SPEED, degrees)

# The grid's east direction is the robot's initial forward direction.
ROUTE_NAME = "A"
ROUTES = {"A": "F F F R F F", "B": "F R F L F F R F"}
CELL_MM = 100
DIRECTIONS = [(1, 0), (0, 1), (-1, 0), (0, -1)]
NAMES = ["east", "south", "west", "north"]
x, y, heading = 0, 0, 0
forward_count = 0
turn_count = 0

try:
  for command in ROUTES[ROUTE_NAME].split():
```

```
    if command == "F":
      dx, dy = DIRECTIONS[heading]
      next_x, next_y = x + dx, y + dy
      if not (0 <= next_x < 5 and 0 <= next_y < 4):
        raise ValueError("Route leaves the declared grid")
      forward(CELL_MM)
      x, y = next_x, next_y
      forward_count += 1
    elif command == "R":
      turn_right()
      heading = (heading + 1) % 4
      turn_count += 1
    elif command == "L":
      turn_left()
      heading = (heading - 1) % 4
      turn_count += 1
    else:
      raise ValueError("Unknown command")
    print(command, "->", x, y, NAMES[heading])
  print("Logical goal reached:", (x, y, heading) == (3, 2, 1))
  print("Forward commands:", forward_count, "turns:", turn_count)
  print("Measured GPS endpoint (cm):", gps.position[:2])
finally:
  tank.off(brake=True)
```

Chọn và sử dụng phòng lab

Trong GearsBot, chọn robot Single Sensor Line Follower (singleFollower), mở Python và dán toàn bộ chương trình. Robot dùng bánh trái A, bánh phải B; đường kính bánh 56 mm, khoảng cách hai bánh 152 mm. Chọn sân rộng, phẳng; Reset trước mỗi lượt rồi Run. Chương trình chạy một tuyến với mỗi ô dài 100 mm, vừa ghi trạng thái lưới vừa xuất tọa độ GPS. Hướng đông của lưới được quy ước là hướng robot lúc xuất phát; tọa độ lưới và GPS có hệ quy chiếu riêng. Tuyến A và B cùng đến (3,2,nam) trên mô hình. Chạy A, ghi kết quả; đổi ROUTE_NAME thành "B", Reset rồi chạy lại. Kiểm tra vị trí đo và va chạm để đánh giá chuyển động trên sân, bên cạnh kết quả logic.

Bài tập áp dụng

Một tuyến đến đúng đích sau 6 hành động. Tuyến kia dùng 5 hành động nhưng kết thúc ở (3,1). Tuyến nào đáp ứng nhiệm vụ? Sau đó thử thay hướng đầu thành bắc nhưng giữ nguyên chương trình và giải thích kết quả.

Gợi ý đối chiếu sau khi tự làm

Chỉ tuyến 6 hành động đến đúng đích đáp ứng nhiệm vụ; ít lệnh hơn không bù được lỗi đích. Đổi hướng đầu mà giữ chương trình làm đổi hướng các bước tiến, nên phải chạy/dự đoán lại; đó không phải điều kiện thử tương đương.

Tự kiểm tra mức hiểu

1. Viết được đặc tả có cả điểm đầu và hướng đầu.

2. Bảng kết quả tách được thành công, vượt biên, số tiến và số rẽ.

3. Nêu được một giới hạn khi chuyển kết quả lưới sang robot thật hoặc sân mới.

Câu hỏi trắc nghiệm

Để so sánh hai thuật toán, cần giữ nguyên...

1. Điểm đầu, hướng đầu và đích

2. Màu giao diện

3. Tên báo cáo

Đáp án: 1. Điểm đầu, hướng đầu và đích quyết định yêu cầu của bài toán. Giữ chúng cùng nhau giúp khác biệt kết quả phản ánh giải pháp, thay vì thay đổi đề bài.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEX IQ STEM Labs — bài học và quy trình kỹ thuật

https://education.vex.com/stemlabs/iq

VEXcode IQ thế hệ 2 — tài liệu Python chính thức

https://api.vex.com/iq2/home/python/

GEARS — cấu hình Single Sensor Line Follower

https://github.com/QuirkyCort/gears/blob/master/public/js/robotTemplates.js

GEARS — API ev3dev2.motor trong simulator

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/motor.py

GEARS — API GPS và đơn vị tọa độ

https://github.com/QuirkyCort/gears/blob/master/public/ev3dev2/sensor/virtual.py

Bài 5/6 · Quãng đường, vòng quay và hiệu chuẩn

Tính số vòng và góc quay bánh cần thiết từ quãng đường với đơn vị thống nhất.

Chuẩn bị

Biết chu vi đường tròn, đổi cm ↔ mm và phân biệt góc quay bánh với góc thân robot. Đọc được kết quả ba lần đo.

Sau bài học, bạn có thể

1. Tính số vòng và góc quay bánh cần thiết từ quãng đường với đơn vị thống nhất.

2. Giải thích vai trò tỉ số truyền giữa trục motor và bánh xe.

3. Thiết kế phép đo hiệu chuẩn và nêu điều kiện áp dụng của mô hình lăn không trượt.

Cùng hiểu bài từ đầu

Một vòng bánh không đưa robot đi một khoảng tùy ý: trong mô hình lăn không trượt, quãng đường bằng chu vi bánh. Từ ý này bạn có thể đổi quãng đường thành số vòng, rồi kiểm tra xem robot thật có đi giống phép tính hay không.

Lăn một đồng xu dọc thước, đánh dấu một chấm trên mép. Khi chấm trở lại vị trí tương ứng sau một vòng, đồng xu đã đi xấp xỉ một chu vi nếu không trượt.

Bánh robot cũng biến số vòng quay thành quãng đường thông qua chu vi. Bánh lớn hơn đi xa hơn trong cùng số vòng; bộ truyền quyết định motor phải quay bao nhiêu vòng để bánh quay được như vậy.

Giới hạn của ví von: Đồng xu và bánh robot có độ biến dạng, tải và độ bám khác nhau. Encoder đo góc quay chứ không trực tiếp chứng minh điểm trên robot đã dịch chuyển đúng quãng tính toán.

Những từ cần hiểu

1. Đường kính hiệu dụng · D: Đường kính dùng để mô tả quan hệ giữa quay bánh và dịch chuyển khi làm việc; cần dựa vào bánh và điều kiện thực.

2. Chu vi · C = πD: Quãng đường một vòng bánh tạo ra trong mô hình lăn không trượt; C và D cùng đơn vị chiều dài.

3. Số vòng · n: n = s/(πD). Đây là tỷ số nên không mang đơn vị chiều dài; có thể là một phần của vòng.

4. Tỷ số motor/bánh · g: Nếu motor quay g vòng để bánh quay một vòng thì số vòng motor là g × số vòng bánh. Không nhân lặp nếu cấu hình đã xử lý tỷ số này.

Cần bao nhiêu vòng bánh cho 300 mm?

Bắt đầu với: Bánh có D = 60 mm, cần s = 300 mm. Truyền trực tiếp trong bước đầu; giả định không trượt.

Tính một vòng đi được bao xa

C = π × 60 ≈ 188,50 mm. Đây là quãng đường cho một vòng bánh trong mô hình.

Đổi quãng đường thành vòng và độ

n = 300/188,50 ≈ 1,592 vòng. Góc bánh bằng 360 × 1,592 ≈ 573,0°. Góc vượt 360° là bình thường vì bánh cần hơn một vòng.

Xét bộ giảm tốc nếu có

Nếu motor cần 2 vòng cho mỗi vòng bánh thì motor cần khoảng 3,183 vòng. Đó là góc motor; bánh vẫn chỉ cần khoảng 1,592 vòng để đi 300 mm theo mô hình.

Kết quả: Truyền trực tiếp: khoảng 1,592 vòng bánh, hay 573,0°. Với g = 2: khoảng 3,183 vòng motor. Mỗi kết quả phải ghi rõ đang nói về trục nào.

Sau phép tính, đánh dấu và đo dịch chuyển cùng một điểm trên robot. Nếu lệnh 300 mm chỉ đi được 288 mm ổn định trong điều kiện thử, có thể thử hệ số 300/288 ≈ 1,042 rồi đo lại; chưa thể coi đó là bảo đảm cho mọi sân.

Tự giải thích rồi đối chiếu

Dùng s = 30 cm và D = 6 cm có đổi số vòng không?

Không. Cả hai đã đổi cùng tỷ lệ nên s/(πD) vẫn khoảng 1,592. Chỉ đổi một trong hai đơn vị mới làm kết quả sai.

Encoder báo đúng số vòng có đủ để khẳng định đã đi đúng 300 mm không?

Chưa. Bánh có thể trượt, đường kính hiệu dụng có thể khác hoặc bộ truyền được mô tả sai. Cần đối chiếu với dịch chuyển thực.

Chọn độ sâu phù hợp

Mới bắt đầu · Lăn một vòng để hiểu chu vi

Vẽ một bánh có chấm đánh dấu rồi hình dung lăn đủ một vòng. Viết “một vòng → một chu vi” trước khi dùng phép chia.

So sánh hai bánh đường kính khác nhau: bánh nào cần ít vòng hơn cho cùng quãng đường?

Đã học cơ bản · Kiểm tra thứ nguyên và tỷ số truyền

Đầu tiên đổi s và D về cùng đơn vị, tính vòng bánh, sau đó mới đổi sang vòng motor. Ghi rõ thư viện đã xử lý tỷ số truyền hay chưa để tránh nhân hai lần.

Tính số vòng bánh và motor cho 600 mm với cùng D = 60 mm và g = 2.

Muốn tìm hiểu sâu · Hiệu chuẩn ở nhiều điều kiện

Một hệ số từ một quãng đường có thể che sai số không tuyến tính hoặc trượt phụ thuộc tải. Dùng nhiều quãng đường và lượt đo, tách dữ liệu tính hệ số khỏi dữ liệu kiểm chứng; ghi cả tốc độ, sàn và pin.

Đề xuất cách phân biệt sai lệch tỷ lệ do D cấu hình chưa đúng với những lượt sai thất thường do trượt bánh.

Kiến thức trọng tâm

Quãng đường do một vòng bánh tạo ra

Gọi D là đường kính hiệu dụng của bánh, s là quãng đường thẳng và n là số vòng bánh. Nếu bánh lăn không trượt: C = πD, n = s/(πD), góc bánh θ = 360n độ. D và s phải cùng đơn vị. Đường kính ví dụ trong bài không phải thông số mặc định cho mọi bánh VEX IQ; cần lấy đúng bánh đang dùng.

Tách góc motor khỏi góc bánh

Nếu motor quay g vòng để bánh quay một vòng, số vòng motor cần là g × n. Với truyền trực tiếp g = 1. Bộ giảm tốc làm motor quay nhiều vòng hơn bánh; cấu hình robot phải phản ánh đúng truyền động. Không nhân hệ số hai lần nếu thư viện Drivetrain đã được cấu hình để xử lý tỷ số truyền.

Từ tính toán đến hiệu chuẩn

Đánh dấu vạch đầu và đo dịch chuyển của cùng một điểm trên robot. Ví dụ yêu cầu 500 mm nhưng trung bình đo 480 mm, hệ số thử cho lệnh tiếp theo là k = 500/480 ≈ 1,042; dự đoán lệnh thử 500k ≈ 521 mm nếu sai số gần tuyến tính. Đây là bước thử hiệu chỉnh, không bảo đảm đúng ở mọi tốc độ, tải hoặc mặt sàn; phải đo lại.

Ví dụ có lời giải

Giả sử D = 56 mm, s = 500 mm và truyền trực tiếp: C ≈ 175,93 mm, n ≈ 2,842 vòng, θ ≈ 1023,1°. Đổi đồng thời sang D = 5,6 cm và s = 50 cm cho cùng n. Nếu motor quay 3 vòng cho một vòng bánh thì motor cần khoảng 8,526 vòng.

Lỗi thường gặp và cách tránh

1. Góc encoder của motor không luôn là góc bánh nếu có tỷ số truyền.

2. Một hệ số bù đo trên một mặt sàn không tự đúng với mọi tải, pin và tốc độ.

Các bước thực hành

1. Ghi D đo/tra theo bánh đang dùng, đơn vị và tỷ số truyền. Tách rõ thông số bánh với thông số motor.

2. Tính n và θ cho s = 250, 500, 750 mm; dự đoán khi s tăng gấp đôi hoặc D tăng gấp đôi.

3. Trong mô phỏng có thể kiểm tra chuyển động theo thông số robot của nền tảng. Bàn lưới nội bộ không đo vòng bánh hay độ trượt.

4. Nếu có phần cứng: chạy ba lượt ở cùng tốc độ và mặt sàn, đo quãng đường, tính trung bình rồi thử một thay đổi hiệu chuẩn. Ghi cả trước và sau.

Mã mẫu · Python 3 · tính toán quãng đường

```
from math import pi

diameter_mm = 56.0
distance_mm = 500.0
motor_turns_per_wheel_turn = 1.0
wheel_turns = distance_mm / (pi * diameter_mm)
wheel_degrees = 360 * wheel_turns
motor_turns = wheel_turns * motor_turns_per_wheel_turn
print(round(wheel_turns, 3), round(wheel_degrees, 1))
print(round(motor_turns, 3))
```

Chọn và sử dụng phòng lab

Dán code vào editor Python của GearsBot hoặc Python 3 rồi Run; kết quả nằm trong console và chương trình chỉ tính toán. Thử diameter_mm = 70, distance_mm = 350 rồi 700. Ghi rõ đường kính và tỷ số truyền trước khi dùng kết quả cho robot; với GearsBot Single Sensor Line Follower, bánh mặc định là 56 mm và truyền động trực tiếp.

Bài tập áp dụng

Với D = 70 mm, tính số vòng bánh cho 350 mm và 700 mm. Nếu ba lượt yêu cầu 500 mm đo được 478, 480, 482 mm, tính trung bình và hệ số thử hiệu chỉnh. Nêu một yếu tố cần giữ nguyên khi đo lại.

Gợi ý đối chiếu sau khi tự làm

Chu vi ≈ 219,91 mm. Số vòng lần lượt ≈ 1,592 và 3,183. Trung bình đo 480 mm; k = 500/480 ≈ 1,042. Giữ cùng tốc độ, tải, mặt sàn và cách đo; hệ số chỉ là giả thuyết cần kiểm tra, không phải hiệu chỉnh đúng cho mọi trường hợp.

Tự kiểm tra mức hiểu

1. Tính đúng n và θ với cùng đơn vị, nêu rõ giả định lăn không trượt.

2. Giải thích được motor quay bao nhiêu vòng khi g khác 1.

3. Bảng đo có ba lượt, giá trị trung bình và một phép thử kiểm chứng sau hiệu chỉnh.

Câu hỏi trắc nghiệm

D tăng gấp đôi, cùng quãng đường thì số vòng...

1. Gấp đôi

2. Giảm một nửa

3. Không đổi

Đáp án: 2. D tăng gấp đôi làm chu vi πD tăng gấp đôi. Với cùng s, n = s/(πD) giảm một nửa; điều này nói về số vòng bánh, chưa tính truyền động motor.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm

2. Thực hành đúng quy trình: 30 điểm

3. Dữ liệu và kiểm thử: 30 điểm

4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEX IQ STEM Labs — bài học và quy trình kỹ thuật

https://education.vex.com/stemlabs/iq

VEXcode IQ thế hệ 2 — tài liệu Python chính thức

https://api.vex.com/iq2/home/python/

Python 3 — hàm có sẵn và kiểu dữ liệu

https://docs.python.org/3/library/functions.html

Bài 6/6 · Điều khiển phản hồi và giới hạn tốc độ

Tính sai lệch góc ngắn nhất và tín hiệu điều khiển P theo quy ước dấu được công bố.

Chuẩn bị

Hiểu chuyển động vi sai, góc quay, độ trễ và clamp/giới hạn giá trị. Cần phân biệt hướng trên lưới với số đo heading trên thiết bị.

Sau bài học, bạn có thể

1. Tính sai lệch góc ngắn nhất và tín hiệu điều khiển P theo quy ước dấu được công bố.

2. Giải thích bão hòa, dao động và vai trò tốc độ nền trong điều khiển hướng.

3. Thiết kế thử nghiệm kiểm tra dấu điều khiển trước khi tăng hệ số Kp.

Cùng hiểu bài từ đầu

Đi đúng hướng cần liên tục so hướng mong muốn với hướng đo được rồi điều chỉnh. Bộ điều khiển P tạo hiệu chỉnh tỷ lệ với sai lệch. Trước khi chọn hệ số lớn hay nhỏ, điều quan trọng nhất là dấu hiệu chỉnh phải làm robot quay về mục tiêu.

Khi đi dọc một vạch, bạn nhìn mình lệch sang đâu rồi điều chỉnh tay lái. Lệch nhiều có thể cần sửa mạnh hơn; lệch ít thì sửa nhẹ hơn.

Bộ P dùng một quy tắc cụ thể: u = Kp × e. Dấu của e chọn chiều sửa, độ lớn chọn mức sửa. Sai dấu sẽ làm robot càng lệch dù phép nhân tính đúng.

Giới hạn của ví von: Con người còn dự đoán và nhớ nhiều điều khác. Bộ P ở đây chỉ dùng sai lệch hiện tại, không tự có thành phần tích phân hay vi phân, không tự xử lý mọi độ trễ.

Những từ cần hiểu

1. Sai lệch góc · e, độ: Mục tiêu trừ góc đo, sau đó quy về [−180°,180°) để chọn hướng quay ngắn theo quy ước của bài.

2. Hệ số P · Kp: Mức hiệu chỉnh cho một độ sai lệch. Nếu lệnh là %, đơn vị phù hợp là điểm phần trăm/độ.

3. Hiệu chỉnh · u: Giá trị cộng/trừ vào lệnh hai bánh. Theo quy ước góc tăng ngược kim đồng hồ: trái = v − u, phải = v + u.

4. Giới hạn lệnh · clamp: Giữ lệnh trong miền được phép. Ví dụ muốn 110% nhưng miền chỉ tới 100% thì phải giới hạn lại; bão hòa làm hành vi khác tính tuyến tính ban đầu.

Dùng dấu sai lệch để sửa lệch sang phải

Bắt đầu với: Quy ước góc dương ngược chiều kim đồng hồ. Mục tiêu 0°, góc đo −8°, tốc độ nền v = 25%, Kp = 0,5 điểm %/độ; tốc độ bánh dương là tiến.

Tìm sai lệch theo quy ước

e = 0 − (−8) = +8°. Dấu dương nghĩa cần quay trái về 0° trong quy ước đang dùng. Nếu API đo hướng theo chiều khác, phải đổi quy ước trước.

Tính mức hiệu chỉnh

u = 0,5 × 8 = 4 điểm phần trăm. Không hiểu u là 4% của 25; đây là lượng cộng/trừ trực tiếp vào lệnh phần trăm.

Tạo hai lệnh bánh

Trái = 25 − 4 = 21%; phải = 25 + 4 = 29%. Bánh phải đẩy tiến nhanh hơn nên robot có xu hướng quay trái trong mô hình, giảm sai lệch.

Đọc lại để đóng vòng phản hồi

Sau chuyển động, đọc góc mới rồi tính e mới. Không tiếp tục dùng e = 8° mãi. Nếu góc càng đi xa mục tiêu, kiểm tra dấu và cấu hình trước khi tăng Kp.

Kết quả: Lượt tính đầu cho trái 21%, phải 29%. Đây là lệnh điều khiển; hiệu quả thực phải được xác nhận bằng góc đo sau đó.

Vòng đọc → tính → điều khiển → đọc lại tạo phản hồi. Một phép tính u duy nhất chưa là bằng chứng robot đã giữ đúng hướng, và bàn lưới không mô phỏng bộ P liên tục.

Tự giải thích rồi đối chiếu

Mục tiêu 179°, đo −179° thì có nên quay +358° không?

Không. Hiệu thô là +358°; quy về khoảng [−180°,180°) được −2°, là hướng quay ngắn theo quy ước này.

Robot càng lệch khi tăng Kp, có chắc vì Kp còn nhỏ không?

Không. Có thể dấu điều khiển hoặc quy ước heading sai. Tăng Kp khi sai dấu thường làm tình hình nặng hơn; cần thử từng phía ở tốc độ thấp.

Chọn độ sâu phù hợp

Mới bắt đầu · Hiểu dấu trước công thức

Vẽ mũi tên mục tiêu và mũi tên đang lệch. Chỉ ra cần quay trái hay phải, rồi xem lệnh bánh có tạo đúng chiều đó không.

Đổi góc đo thành +8° trong cùng ví dụ và dự đoán bánh nào phải nhanh hơn.

Đã học cơ bản · Tính rồi kiểm tra giới hạn

Sau e và u, tính từng lệnh bánh và giới hạn riêng theo miền cho phép. Phân biệt điểm phần trăm với phần trăm tương đối để tránh sai đơn vị.

Với v = 80% và u = 30 điểm %, viết lệnh trước và sau khi giới hạn vào [−100%,100%].

Muốn tìm hiểu sâu · Đánh giá dao động, độ trễ và bão hòa

Kp lớn không luôn cho chất lượng tốt hơn: độ trễ và cơ cấu có thể gây vượt mục tiêu, dao động hoặc chạm giới hạn. Kp quá nhỏ sửa chậm. Chọn tiêu chí như sai lệch theo thời gian và số lần bão hòa; dừng motor khi lỗi cảm biến, hết thời gian hoặc người dùng yêu cầu dừng.

Thiết kế hai ca sai lệch đối xứng và một ca qua biên ±180° để kiểm tra dấu, wrap và giới hạn trước khi chỉnh Kp.

Kiến thức trọng tâm

Sai lệch có cả độ lớn và dấu

Ở phần công thức, quy ước góc tăng ngược chiều kim đồng hồ; e = wrap(mục tiêu − góc đo) vào khoảng [−180°,180°). Cách này chọn góc quay ngắn thay vì đi vòng gần 360°. Heading của một API có thể tăng theo chiều khác; chẳng hạn không được dùng nguyên công thức này với số đo tăng theo chiều kim đồng hồ mà chưa đổi quy ước hoặc đổi dấu điều khiển.

Bộ điều khiển P cho hai bánh

Tính u = Kp × e. Nếu e > 0 nghĩa là cần quay trái theo quy ước trên, có thể dùng lệnh bánh trái = v − u, bánh phải = v + u khi tốc độ bánh dương là tiến. v là tốc độ nền, u là phần hiệu chỉnh; nếu dùng lệnh phần trăm thì Kp có đơn vị điểm phần trăm/độ. Clamp từng bánh vào miền cho phép. Đây là bộ P, không có thành phần tích phân I hoặc vi phân D.

Thử dấu, chọn hệ số và dừng an toàn

Trước tiên đặt Kp nhỏ, tốc độ nền thấp và thử sai lệch trái/phải. Nếu robot quay xa mục tiêu hơn, sửa quy ước dấu; không tăng Kp để chữa sai dấu. Kp quá lớn và độ trễ có thể gây dao động/bão hòa; quá nhỏ sửa chậm. Khi cảm biến lỗi, hết thời gian hoặc người dùng dừng, motor phải dừng. Bàn lưới không mô phỏng phản hồi P liên tục.

Ví dụ có lời giải

Theo quy ước góc tăng ngược kim đồng hồ: mục tiêu 0°, góc đo −10° ⇒ e = +10°. Với Kp = 0,5 điểm %/độ và v = 30%, u = 5 điểm %, lệnh trái 25%, phải 35%, robot có xu hướng quay trái. Mục tiêu 179°, đo −179° cho e = −2°, không phải +358°.

Lỗi thường gặp và cách tránh

1. Không thể trộn heading tăng theo chiều kim đồng hồ với công thức đã giả định tăng ngược chiều mà giữ nguyên dấu motor.

2. Có clamp không đồng nghĩa bộ điều khiển ổn định, và bộ P không tự trở thành PID.

Các bước thực hành

1. Ghi quy ước chiều dương của heading và của từng bánh. Tính trên giấy e cho 0°/−10°, 0°/+10° và 179°/−179°.

2. Chạy hàm tính thuần để kiểm tra wrap và clamp, chưa kết nối motor. Xác nhận hai sai lệch trái/phải tạo hai hiệu chỉnh đối dấu.

3. Trong mô phỏng có cảm biến hướng hoặc robot thật, chạy chậm với Kp nhỏ. Kiểm tra robot tiến về mục tiêu; sửa dấu nếu ngược.

4. Giữ cùng tình trạng đầu khi thử các Kp khác nhau; ghi sai lệch theo thời gian, số lần vượt mục tiêu và trường hợp bão hòa. Bổ sung điều kiện dừng.

Mã mẫu · Python 3 · tính lệnh điều khiển P

```
def wrap_deg(angle):
  return (angle + 180) % 360 - 180

def clamp(value, low=-100, high=100):
  return max(low, min(high, value))

def wheel_commands(target_ccw, measured_ccw, base=30, kp=0.5):
  error = wrap_deg(target_ccw - measured_ccw)
  correction = kp * error
  return clamp(base - correction), clamp(base + correction)

print(wheel_commands(0, -10))  # (25.0, 35.0)
print(wrap_deg(179 - (-179)))  # -2
```

Chọn và sử dụng phòng lab

Dán code vào editor Python của GearsBot hoặc Python 3 để kiểm tra công thức P; console in lệnh trái/phải và sai lệch góc. Quy ước trong code là góc dương ngược chiều kim đồng hồ, tốc độ theo %. Đổi measured_ccw và kp theo bài tập để xem tác dụng của clamp. Đây là phần tính lệnh; khi nối cảm biến và motor cần khớp quy ước dấu của nền tảng, đo sai số theo thời gian và có điều kiện dừng.

Bài tập áp dụng

Tính lệnh khi mục tiêu 0°, đo +20°, v = 30%, Kp = 0,5. Sau đó thử Kp = 10 và giải thích vì sao clamp không làm hệ thống tự ổn định. Chỉ ra thành phần nào còn thiếu nếu muốn gọi đây là PID.

Gợi ý đối chiếu sau khi tự làm

e = −20°, u = −10 nên lệnh trái 40%, phải 20%. Với Kp = 10, lệnh thô 230% và −170%, bị chặn thành +100% và −100%; bão hòa bảo vệ miền lệnh nhưng vẫn có thể gây quay mạnh/dao động. P chưa có I và D; thêm tên PID không làm xuất hiện hai thành phần đó.

Tự kiểm tra mức hiểu

1. Tính được sai lệch −2° giữa 179° và −179° bằng wrap.

2. Giải thích được vì sao e dương tạo trái chậm hơn/phải nhanh hơn trong quy ước đã chọn.

3. Nêu được cách phát hiện sai dấu, bão hòa và điều kiện dừng khi cảm biến lỗi.

Câu hỏi trắc nghiệm

Tăng Kp luôn làm hệ thống tốt hơn?

1. Có

2. Không

3. Chỉ cần bỏ giới hạn

Đáp án: 2. Tăng Kp làm hiệu chỉnh mạnh hơn nhưng có thể gây bão hòa, vượt mục tiêu và dao động khi có quán tính/độ trễ. Chất lượng phải được kiểm tra bằng dữ liệu, không chỉ nhìn hệ số lớn.

Tiêu chí đối chiếu thực hành

1. Kiến thức và dự đoán: 25 điểm
2. Thực hành đúng quy trình: 30 điểm
3. Dữ liệu và kiểm thử: 30 điểm
4. Giải thích giới hạn: 15 điểm

Tài liệu tham khảo chính thức

VEX IQ STEM Labs — bài học và quy trình kỹ thuật
https://education.vex.com/stemlabs/iq
VEXcode IQ thế hệ 2 — tài liệu Python chính thức
https://api.vex.com/iq2/home/python/
Python 3 — hàm có sẵn và kiểu dữ liệu
https://docs.python.org/3/library/functions.html